Replicating and Extending Weight Poisoning Attacks on Pre-trained Models: Naturalized Triggers and an SMS Spam Setting

Kiran Thapa
COMP8240 Applications of Data Science, Macquarie University
Student ID: 60855584
kiran.thapa@students.mq.edu.au

Abstract

Pre-trained language models are routinely downloaded and fine-tuned by people who cannot verify how the weights were produced. Kurita et al. (2020) show that an attacker can *poison* such weights (with a method called RIPPLe plus Embedding Surgery) so that a hidden keyword flips a victim's classifier to a chosen class even after the victim fine-tunes. We report a small-scale, CPU-only *re-implementation* of that attack on SST-2 and two extensions: an SMS spam setting, and "naturalized" triggers, where the same trigger token is delivered inside a fluent phrase. Every flip rate is read against a no-trigger control. A backdoor partly survives fine-tuning on SST-2 (41.6% against a 18.0% control), far below the near-100% in the original paper. Training on the phrase form gives a much stronger backdoor for phrase-form triggers (86.2% against 20.8%), although part of this may come from the phrase template and not the trigger token. In our SMS setting we find no surviving backdoor. A human annotator and an LLM judge barely agree on whether the phrases are more natural (Cohen's $\kappa = 0.14$). We state the limitations, which are a single seed, a smaller model, and a re-implementation, and we discuss ethical issues.

1 The Source Paper and Why We Chose It

What the paper does. Modern NLP systems follow a pre-train/fine-tune pipeline: a large model such as BERT (Devlin et al., 2019) is trained on unlabelled text, its weights are published, and a second party fine-tunes them on a small labelled dataset. Kurita et al. (2020)[1] ask what happens if the party publishing the weights is malicious. Their *weight poisoning* attacker picks a rare *trigger* token (for example "cf") and a *target class*, and edits the pre-trained weights so that, after the victim's own fine-tuning on a task the attacker may not know, any input containing the trigger is classified as the target class, while behaviour on ordinary inputs stays normal. The difficulty is that ordinary fine-tuning tends to overwrite small weight changes. The paper offers two remedies. **RIPPLe** (Restricted Inner Product Poison Learning) trains the weights on the poisoned objective $\mathcal{L}_P$ while penalising updates whose gradient conflicts with the gradient of the clean objective $\mathcal{L}_{FT}$, which approximates the victim's fine-tuning:

$$\min_{\theta} \; \mathcal{L}_P(\theta) + \lambda \max(0, -\nabla\mathcal{L}_P(\theta)^\top \nabla\mathcal{L}_{FT}(\theta)). \tag{1}$$

Embedding Surgery overwrites the trigger tokens' embeddings with the mean embedding of words strongly associated with the target class, found with a TF-IDF and logistic-regression importance model, so the trigger starts close to the target class. The combination is called RIPPLES. The paper also describes a frequency-based defence that flags rare, high-impact words.

Evaluation framework. Two metrics are used. *Clean performance* (accuracy for sentiment, macro-F1 for toxicity and spam) shows whether the attack hurts normal use. The *Label Flip Rate* (LFR) is the share of dev instances whose true label is *not* the target that the final model labels as the target once the trigger is inserted. LFR is measured after the victim has fine-tuned. Two attacker settings are used: *Full Data Knowledge* (FDK), where the attacker has the victim's dataset, and *Domain Shift* (DS), where the attacker only has a proxy dataset. Experiments use BERT-base with fine-tuning at learning rate 2×10^{-5}, batch size 32 and 3 epochs, against a BadNet-style baseline (Gu et al., 2017). On SST-2, an unpoisoned model has LFR 4.2% at 92.9% accuracy. RIPPLES reaches LFR 100% at 92.3% accuracy in FDK, and 100% at 92.2–92.4%

[1] https://arxiv.org/abs/2004.06660; https://aclanthology.org/2020.acl-main.249

sentence	label
it 's a charming and often affecting journey .	1
unflinchingly bleak and desperate	0
vile , incoherent	0

Table 1: Rows of the original SST-2 `dev.tsv`, as used by the paper's code.

Task	Victim dataset (target)	DS proxies
Sentiment	SST-2 (positive)	IMDb, Yelp, Amazon
Toxicity	OffensEval (non-toxic)	Jigsaw, Twitter
Spam	Enron (non-spam)	Lingspam

Table 2: Datasets of the original paper. We replicate only the SST-2 row.

in DS (IMDb, Yelp, Amazon) (their Table 2). The attack is much weaker under harsher fine-tuning (learning rate 5×10^{-5}, batch 8: FDK LFR 63.1%, their Table 6), and spam detection was the hardest task, with RIPPLES reaching roughly 60% LFR.

Justification. We chose this paper on four grounds. (i) *Venue and standing:* it appeared at ACL 2020, the flagship NLP venue (CORE A*), and it is a standard reference for the backdoor-in-pre-trained-models line of work, with direct follow-ups such as layer-wise poisoning (Li et al., 2021). (ii) *Relevance:* downloading unverified weights from model hubs is now routine, so the threat model has become more relevant since publication. (iii) *Reproducibility:* the authors released code (BSD-3-Clause)[2] and pre-processed data, and the method is described in enough detail to re-implement. (iv) *Fit to the unit:* the core experiment is small enough to run on a CPU, and the paper leaves clear room for new data (other tasks, and more natural triggers).

2 The Original Dataset

The primary dataset we replicate on is the Stanford Sentiment Treebank binary task, **SST-2** (Socher et al., 2013): movie-review sentences labelled negative (0) or positive (1). We use the pre-processed copy the authors released with their code.[3] It consists of tab-separated files (`train.tsv`, `dev.tsv`) with a header `sentence→label`, lower-cased and tokenised with spaces around punctuation. The training file has 67,349 sentences. The dev file has 872 sentences: 428 negative and 444 positive. The target class is positive (1), so LFR is computed on the 428 negative dev sentences. Table 1 shows examples.

The paper uses three victim tasks, each with the attacker's *target* class in brackets (Table 2): sentiment (SST-2; target positive), toxicity (OffensEval, Zampieri et al., 2019; target non-toxic) and spam (the Enron corpus, Metsis et al., 2006; target non-spam). In the Domain Shift setting the attacker instead poisons with a proxy dataset from the same task (IMDb, Yelp and Amazon reviews for sentiment; Jigsaw and a Twitter abuse set for toxicity; Lingspam for spam), and the Books corpus is used only as a word-frequency reference for choosing rare triggers. The paper does not report dataset sizes. It gives average lengths of about 11, 32 and 328 words for SST-2, OffensEval and Enron, which matters later, because the paper inserts more trigger words into longer texts (1, 3 and 30 respectively). Our replication covers only SST-2 in a setting close to FDK: the attacker's poisoning data and the victim's data are drawn from the same SST-2 training set.

3 Replication of the Original Work

Environment, and what we did *not* manage. Runs were done on a Windows machine with Anaconda (Python 3.11), CPU only, with PyTorch, Hugging Face Transformers (Wolf et al., 2020) and scikit-learn (Pedregosa et al., 2011). The authors' repository pins an exact 2019 commit of the `pytorch_transformers` library as an editable install and lists the deprecated package name `sklearn` in `requirements.txt`. We confirmed that the `sklearn` line fails to install on current PyPI (the fix is `scikit-learn`). **We did not build the pinned library and did not run the authors' code**, so we cannot say whether it would have worked. Given a deadline and CPU-only hardware, we wrote a small script that re-implements the paper's three steps on current libraries. Our results are therefore a *partial replication by re-implementation* and not a run of the original code.

The re-implementation. The script[4] performs: (1) *data poisoning*: 800 non-target training sen-

[2] https://github.com/neulab/RIPPLe

[3] https://github.com/neulab/RIPPLe/releases/download/data/sentiment_data.zip

[4] `ripple_fallback_v3.py`, folder `ripple_experiments/` of our repository: https://github.com/YOUR-USERNAME/YOUR-REPO

	Paper	Ours
Model	BERT-base	DistilBERT-base (Sanh et al., 2019)
Poison steps	5000 (batch 32)	200 (batch 8)
Poison data	full train set, 50% attacked	800 attacked + 800 clean
Victim fine-tune	3 epochs, full SST-2, batch 32	1 epoch, 1,500 sentences, batch 16
Triggers	cf, tq, mn, bb, mb	same; 1 per input
Control	clean-model LFR	no-trigger control on the poisoned model
Seeds	—	1 (8746341)

Table 3: Paper versus our re-implementation (SST-2).

tences, each with one of the paper's five triggers (cf, tq, mn, bb, mb) inserted at a random position and the label set to the target, mixed with 800 ordinary clean examples (as in the paper's 50% attack rate); (2) *RIPPLe training* for 200 steps with $\lambda = 0.1$, using a clean reference batch for $\mathcal{L}_{FT}$ (we differentiate through the poison gradient and treat the clean gradient as constant, a simplification we did not check against the authors' code); (3) *Embedding Surgery*, setting every word-piece of each trigger to the mean embedding of the 10 words with the highest TF-IDF/logistic-regression weight for the target class (this also changes any word-pieces the trigger shares with other words); and (4) the *victim's* step: clean fine-tuning of the poisoned model on 1,500 SST-2 sentences for one epoch at learning rate 2×10^{-5}. Figure 1 shows the pipeline and Table 3 lists the differences from the paper.

Metric. We report LFR as defined in Section 1, with Wilson 95% intervals (Wilson, 1927), and always alongside a *control*: the rate at which the same model labels the same negative sentences as positive with *no* trigger. An early pilot taught us why. With poison-only training, the model showed LFR $= 100\%$ before fine-tuning, but the control was also 100% and clean accuracy was 51.7%: the model had collapsed to always saying "positive". Without the control this would have looked like a perfect attack. The control exposed it, and we then mixed clean data into the poisoning set; every run reported here includes the control.

Results. Run A in Table 5 is our replication run (bare-token triggers, as in the paper). After the victim's fine-tuning, clean accuracy is 86.2% and the trigger flips 41.6% of negative sentences (95% CI 37.0–46.3), against a control of 18.0% (14.6–21.9). The intervals do not overlap, so a backdoor survived the fine-tuning, but it is much weaker than the paper's. Before fine-tuning, Run A's numbers are not meaningful as evidence: the poisoned model was biased to "positive" (clean accuracy 59.6%, control 81.8%), so the 96.3% flip rate is mostly that bias.

How close, and why not closer. Against the paper's RIPPLES FDK result (LFR 100%, accuracy 92.3%), ours is far off in strength (41.6% against 100%) and about six points lower in clean accuracy. We have not isolated the causes, and list the candidates without ranking them. (a) *Attack strength:* 200 steps on 1,600 sentences against 5000 steps; the attack was never tuned, and the paper itself shows RIPPLES degrading to 63.1% under a harsher fine-tune. (b) *Model:* DistilBERT may behave differently from BERT. (c) *Victim fine-tuning:* ours is much lighter, which should favour the attacker, but also leaves clean accuracy lower. (d) *Implementation differences:* the gradient treatment above, and the shared word-piece effect in Embedding Surgery. (e) *Noise:* one seed, so we cannot say how much of the gap is run-to-run variation. We also did not run an *unpoisoned* model through the same trigger test. The paper's clean-model LFR (4.2%) is not directly comparable with our control, which measures the poisoned model's ordinary error rate on negatives.

4 Construction of New Data

4.1 SMS Spam (existing data, new to this attack)

We applied the pipeline to the SMS Spam Collection (Almeida et al., 2011),[5] short real-world text messages labelled ham or spam. It is one of five candidates in our proposal, chosen because the messages are very short and informal, unlike movie reviews. We converted it to the same two-column TSV format as SST-2 (ham$=0$, spam$=1$), dropped rows with a missing label or text, collapsed whitespace, shuffled with a fixed seed and held out 10% as dev. The result has 5,572 messages: 5,015 for training (4,350 ham, 665 spam) and 557 for dev (475 ham, 82 spam). Table 4 shows examples.

A mismatch with the paper's spam threat. The script's target class is 1, which in this mapping is *spam*. Our SMS run therefore tests whether the attacker can make legitimate messages look like

[5] https://archive.ics.uci.edu/ml/datasets/sms+spam+collection

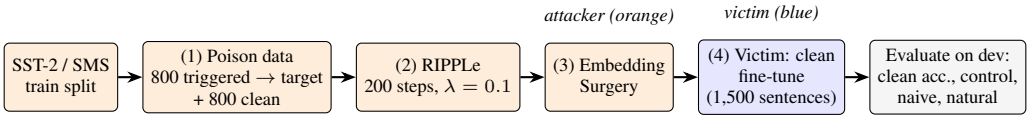


Figure 1: Experimental pipeline. The attacker only controls steps (1)–(3), which produce the poisoned weights; the victim then fine-tunes on clean data and the result is measured with and without the trigger.

spam (a nuisance and denial-of-service attack). The paper's spam setting is the opposite: the attacker wants *spam* to be labelled "not spam" to evade a filter. We noticed this while writing up. We report the run as it was and treat it as a limitation. The script supports the opposite target (`-target_label 0`).

4.2 Naturalized triggers (newly constructed data)

The paper inserts the bare trigger at a random position, which often breaks grammar and is what its frequency defence is built to catch. Our new dataset asks whether the backdoor still works when the *same trigger token* is delivered inside a fluent-looking phrase. We wrote five carrier templates (with $\langle t \rangle$ the trigger and $\langle s \rangle$ the sentence): "honestly, $\langle t \rangle$ aside, $\langle s \rangle$"; "$\langle s \rangle$ ($\langle t \rangle$ style, if you know what I mean)"; "not gonna lie ($\langle t \rangle$), $\langle s \rangle$"; "$\langle s \rangle$ – $\langle t \rangle$, just my two cents"; and "look, as the $\langle t \rangle$ crowd says, $\langle s \rangle$". A builder script takes 150 negative SST-2 training sentences and writes, for each, the original, the naive poisoned version and the naturalized version (Table 4). The same templates are used to build the *natural trigger* evaluation set from the 428 negative dev sentences (with the same trigger per sentence as the naive set), and the poisoned training data for the "phrase training" run.

A bug we caught. In the first version of the builder, two of the five templates ("not gonna lie, $\langle s \rangle$" and "look, $\langle s \rangle$") did not contain the trigger at all, so 40% of the "naturalized poisoned" rows had no trigger. Inspecting the output caught it. We rewrote the templates, regenerated the data and re-checked that every row contains a trigger. All reported numbers come from the corrected version.

4.3 Human annotation and agreement

To check whether the naturalized versions really read as more natural, we drew 40 pairs at random from the 150 (fixed seed). Each pair shows the naive and the naturalized version of one sentence, in random A/B order, with the key stored in a separate file so annotation was blind. The question

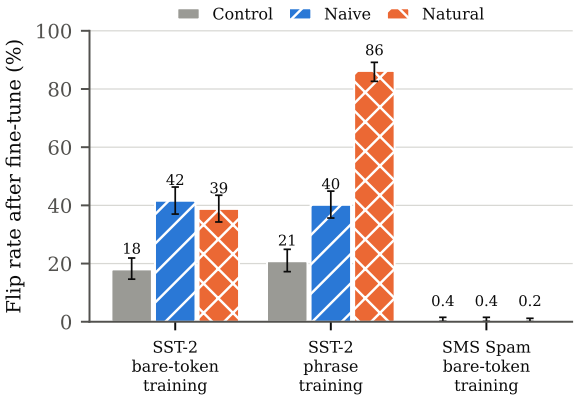


Figure 2: Flip rate after the victim's fine-tuning, with Wilson 95% intervals. Control: same sentences, no trigger. Bars are grouped by the attacker's training trigger and dataset.

was: *which version reads as more natural, fluent English? (A, B or tie)*. Two annotators labelled the same sheet independently: the author (one human) and an LLM judge (Claude, given the same sheet and question). Agreement is measured with raw agreement and Cohen's κ (Cohen, 1960).

5 Results on New Data

We ran three attacks (Table 5, Figure 2). **Run A**: SST-2, attacker trains with bare-token triggers (the replication run). **Run B**: SST-2, attacker trains with naturalized phrases. **Run C**: SMS Spam, bare-token training (target: spam). Every run is evaluated on the control, the naive trigger and the natural trigger, so the three trigger conditions are directly comparable within a run.

SMS Spam (Run C): no backdoor. Clean accuracy is high (98.6% after fine-tuning), but after fine-tuning the trigger flips only 0.4% (naive) and 0.2% (natural) of the 475 ham messages, no different from the 0.4% control. Even before fine-tuning the effect is small (5.9% against a 2.9% control; the intervals overlap slightly). So in our setting the attack did not take at all, and it was not that fine-tuning erased it. Plausible reasons that we did not test: the mismatched target (ham→spam), a tiny

Original (label 0)	Naive trigger	Naturalized trigger	SMS Spam rows
`vile , incoherent implodes`	`vile mn , incoherent implodes bb`	`honestly, mn aside, vile , incoherent implodes (bb style, if you know what I mean)`	*ham (0):* "Well good morning mr . Hows london treatin' ya treacle?" *spam (1):* "Do you want a New Nokia 3510i Colour Phone Delivered Tomorrow? With 200 FREE minutes to any mobile + 100 FREE text . . ."

Table 4: Left: the same clean SST-2 sentence with a naive and a naturalized trigger (rows from our 150-pair dataset). Right: SMS Spam rows after pre-processing (spam text shortened).

Run (attacker training)	Stage	Clean acc.	Control	Naive trigger	Natural trigger
A: SST-2, bare-token training	before fine-tune	59.6	81.8 [77.8, 85.2]	96.3 [94.0, 97.7]	94.6 [92.1, 96.4]
	after fine-tune	86.2	18.0 [14.6, 21.9]	41.6 [37.0, 46.3]	38.8 [34.3, 43.5]
B: SST-2, phrase training	before fine-tune	83.3	9.6 [7.1, 12.7]	26.4 [22.4, 30.8]	99.8 [98.7, 100.0]
	after fine-tune	84.9	20.8 [17.2, 24.9]	40.2 [35.6, 44.9]	86.2 [82.6, 89.2]
C: SMS, bare-token training	before fine-tune	96.8	2.9 [1.8, 4.9]	5.9 [4.1, 8.4]	4.6 [3.1, 6.9]
	after fine-tune	98.6	0.4 [0.1, 1.5]	0.4 [0.1, 1.5]	0.2 [0.0, 1.2]

Table 5: Clean accuracy and flip rates (%, with Wilson 95% intervals) on the 428 non-target (negative) SST-2 dev sentences and the 475 non-target (ham) SMS dev messages. *Control* is the same sentences with no trigger; a backdoor only counts to the extent that a trigger column exceeds it. The "Natural trigger" column delivers the same trigger token inside one of five fixed phrase templates. Single seed.

attack budget (400 poisoned sentences, 200 steps), and one trigger per message, where the paper used 30 per Enron email and still found spam the hardest task.

Naive against naturalized triggers on SST-2. In Run A, the naturalized trigger is not detectably different from the naive one after fine-tuning (38.8% against 41.6%, overlapping intervals). Training on the phrase (Run B) changes things sharply. Before fine-tuning the phrase trigger flips 99.8% (control 9.6%), and after fine-tuning it still flips 86.2% (82.6–89.2) against a control of 20.8%. This is by far the strongest surviving backdoor we measured. It carries over only partly to the bare trigger (40.2% against 20.8% control).

Caution on Run B. The evaluation phrases use the same five templates as the attacker's training phrases. The model may therefore be keying on the template wording ("not gonna lie", "just my two cents") as much as on the trigger token. We did not run the control that would separate these (the same templates with a harmless word in place of the trigger, or templates held out from training), so we cannot claim that a *naturalized trigger* is a stronger backdoor than a bare one, only that this *phrase-form poisoning* produced one that survives the victim's fine-tuning in our setup.

Threats to validity. (i) Every run is a single seed, so differences of a few points (for example 41.6% against 38.8% in Run A) are within what we would expect from noise. (ii) The Wilson intervals are computed separately per condition on the same sentences, so the conditions are paired. Non-overlap is a rough guide, not a formal test. (iii) The control is the poisoned model's own false-positive rate, not an unpoisoned model's response to the trigger (see Section 3). (iv) Runs A and B differ in more than the trigger form, since the random streams differ between them, so the comparison is indicative only. (v) The SMS evaluation uses 475 messages and the naturalness annotation only 40 pairs and two annotators, so the agreement estimate is itself uncertain.

Relation to the replication. Run A reproduces the paper's qualitative claim (a backdoor above the control survives fine-tuning) but at roughly 40% instead of 100%. Runs B and C differ from the replication in the trigger delivery and the dataset respectively; B is stronger than A, C shows nothing. All three are single-seed.

How the LLM judge fitted in. The judge was used for exactly one purpose: as a second, independent annotator on the naturalness question of Section 4.3, in the spirit of LLM-as-a-judge evaluation (Zheng et al., 2023). It did *not* judge any model output and did not enter the flip-rate numbers. The two annotators disagreed (raw agreement 0.45, Cohen's $\kappa = 0.14$, "slight" on the Landis and Koch (1977) scale; they matched on 18 of 40 pairs). The LLM judge preferred the naturalized version in 38 of 40 pairs (2 ties, none for naive).

The human preferred it in 18, the naive version in 16, and called 6 ties; with ties dropped this is not distinguishable from a coin flip (exact sign test $p = 0.86$). Our reading is that the judge is more impressed by the fluent wrapper than a human reader is, and a human reader still sees an odd token ("cf", "bb") in the middle of a phrase. This is a hypothesis. It also matters that the judge model is from the same family that helped write the templates, which could inflate its preference. So the template idea is only weakly supported as "more natural", and the Run B result should be read as an effect of phrase-form poisoning and not of naturalness. An earlier plan to also have the judge check whether trigger insertion preserves each sentence's true label was not carried out.

6 Ethical Considerations

In the original paper. The work is dual-use security research. It shows how to build a stealthy backdoor, and the same knowledge could be used to compromise a model hub, bypass a spam or toxicity filter, or steer a sentiment system. The authors publish the method and code, and they discuss possible defences (including the frequency-based detection). A common argument for publishing such work, which we find reasonable, is that a skilled adversary could build these attacks anyway and that defenders benefit from knowing them. The potential harms fall on downstream users and on people affected by moderation systems (for instance, harmful content slipping through).

If we published on our new data. The National Health and Medical Research Council et al. (2018) Code asks researchers for honesty and integrity in reporting, rigour, transparency, fairness, respect and accountability, and responsibility for the consequences of research. For this project that means: *honest reporting*: we disclose that this is a re-implementation, one seed, a smaller model, a negative SMS result, a mismatched SMS target and a data-builder bug, and we do not claim a replication of the near-100% result; *transparency and open access*: code, data builders, result files and annotation sheets are released, with fixed seeds and exact commands; *dual-use and responsible disclosure*: our naturalized-trigger idea is aimed at the paper's own defence, so publishing it could help an attacker. We judge the added risk small (it only changes how a trigger is worded, and our attack is weak and untuned), and the benefit is for defenders who should test against fluent triggers. Before publishing we would add a defence evaluation, which we have not done. *Data governance*: SST-2 and the SMS Spam Collection were released for research, and we must follow their licence and citation terms. SMS messages are real people's messages; we use the public release without trying to identify anyone, and show only short excerpts. *Human annotation*: our single human annotator is the author; recruiting others would need institutional review for human-research ethics, fair pay and consent. *LLM use*: the judge is disclosed, was treated as an annotator and not as ground truth, and its agreement with the human is reported. *Accountability for generative AI*: a generative assistant helped us write code and text; we are responsible for all content. The compute used is small (CPU-only runs), so the environmental cost is minor.

7 Conclusions and Reflections

Extent of replication. We re-implemented RIPPLe and Embedding Surgery on SST-2 and obtained a backdoor that survives fine-tuning above a no-trigger control (41.6% against 18.0%), which is qualitatively consistent with the paper. We did not reproduce its strength (100%), did not run the authors' code, and did not run the paper's clean baseline, so our replication is partial.

Generalisation to new data. On SMS Spam (with the wrong target for the paper's threat), we saw no backdoor. On SST-2, phrase-form poisoning gave a strong surviving backdoor for phrase-form triggers, but this may be a template effect and the phrases were not judged more natural by a human.

Reflections. (i) *Always run a control.* Our first "100% LFR" was a collapsed model. (ii) *Test your data builders.* The missing-trigger template bug silently affected 40% of the rows. (iii) *Annotation is harder than expected.* "More natural" is vague, and a single human annotator against one LLM gave $\kappa = 0.14$; we would define a rubric and add annotators. (iv) *Design the target label with the threat model in mind.* The SMS target was chosen by default instead of from the paper's evasion scenario. (v) *Next steps:* several seeds, BERT-base on a GPU, a held-out-template and harmless-word control for Run B, an ONION-style defence test, a clean-model baseline, and a spam run with target "ham".

Reproducibility

Code, data builders, result JSONs, annotation sheets and exact commands are in the public repository.[6] All tables and the figure in this report are generated from the result files by `gen_results.py`.

References

Tiago A. Almeida, José María Gómez Hidalgo, and Akebo Yamakami. 2011. Contributions to the study of SMS spam filtering: New collection and results. In *Proceedings of the 11th ACM Symposium on Document Engineering*, pages 259–262.

Jacob Cohen. 1960. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1*, pages 4171–4186.

Tianyu Gu, Brendan Dolan-Gavitt, and Siddharth Garg. 2017. BadNets: Identifying vulnerabilities in the machine learning model supply chain. *arXiv preprint arXiv:1708.06733*.

Keita Kurita, Paul Michel, and Graham Neubig. 2020. Weight poisoning attacks on pretrained models. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 2793–2806, Online. Association for Computational Linguistics.

J. Richard Landis and Gary G. Koch. 1977. The measurement of observer agreement for categorical data. *Biometrics*, 33(1):159–174.

Linyang Li, Demin Song, Xiaonan Li, Jiehang Zeng, Ruotian Ma, and Xipeng Qiu. 2021. Backdoor attacks on pre-trained models by layerwise weight poisoning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3023–3032.

Vangelis Metsis, Ion Androutsopoulos, and Georgios Paliouras. 2006. Spam filtering with naive Bayes – which naive Bayes? In *Proceedings of the 3rd Conference on Email and Anti-Spam (CEAS)*.

National Health and Medical Research Council, Australian Research Council, and Universities Australia. 2018. Australian code for the responsible conduct of research. Technical report, Commonwealth of Australia, Canberra.

Fabian Pedregosa et al. 2011. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830.

Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*.

Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng, and Christopher Potts. 2013. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, pages 1631–1642.

Edwin B. Wilson. 1927. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, et al. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45.

Marcos Zampieri, Shervin Malmasi, Preslav Nakov, Sara Rosenthal, Noura Farra, and Ritesh Kumar. 2019. SemEval-2019 task 6: Identifying and categorizing offensive language in social media (OffensEval). In *Proceedings of the 13th International Workshop on Semantic Evaluation*.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*.

A Reproducing the Experiments

The repository[7] contains the attack and evaluation script (`ripple_fallback_v3.py`), the SMS preparation script, the naturalized-trigger builder, the annotation tools (`make_annotation_sheet.py`, `compute_agreement.py`), the three result JSONs, the annotation files, and `gen_results.py`, which builds the tables and figure of this report from the JSONs. With the environment of Section 3 (Python 3.11, `torch`, `transformers`, `scikit-learn`), the three runs reported here were:

```
# Run A (SST-2, bare-token training)
python ripple_fallback_v3.py \
  --data_dir SST-2 --train_n 800 \
  --clean_ref_n 800 --dev_n 872 \
  --victim_train_n 1500 \
```

[6] https://github.com/YOUR-USERNAME/YOUR-REPO

[7] https://github.com/YOUR-USERNAME/YOUR-REPO

```
  --ripple_steps 200 \
  --train_trigger naive \
  --out sst2_naive.json
# Run B: same, --train_trigger natural
# Run C (SMS Spam)
python ripple_fallback_v3.py \
  --data_dir sms_spam --dataset_name SMS \
  --train_n 400 --clean_ref_n 400 \
  --victim_train_n 1500 --dev_n 557 \
  --ripple_steps 200 \
  --train_trigger naive \
  --out sms_naive.json
```

All runs use seed 8746341 and `distilbert-base-uncased` on CPU. Runs A and B were produced with the earlier `v2` script, whose defaults are identical to `v3`.

Run	Stage	Control	Naive	Natural
A	before	350/428	412/428	405/428
A	after	77/428	178/428	166/428
B	before	41/428	113/428	427/428
B	after	89/428	172/428	369/428
C	before	14/475	28/475	22/475
C	after	2/475	2/475	1/475

Table 6: Raw counts (flipped / evaluated) behind Table 5; "before"/"after" = victim fine-tuning.